**EQUIPO:**

**Héctor Ulises Cacho González**

**Ciro Julián Cervantes Zamora**

**Dana Lizbeth Castañeda Sánchez**

**Concepcion Guadalupe Paniagua Gonzalez**

**Oscar Yael Hernández Rodríguez**

**Docente:**

**José Miguel Carrera Pacheco**

**Materia:**

**Desarrollo Web Profesional**

**Actividad:**

**[ACT] S - 7. Comunicación asíncrona accesible**

**FECHA:**

**20 de febrero de 2026**

# ESTANDAR OFICIAL DE API REST (Backend - Frontend)

## 1. ESTRUCTURA ESTÁNDAR DE ERROR

Cualquier petición que falle (4xx o 5xx) devolverá **siempre** esta misma estructura JSON para que el Frontend la procese fácilmente en su catch:

JSON

```
{
  "success": false,
  "error": "TipoDeError",
  "message": "Mensaje legible para el usuario final (ideal para aria-live)"
}
```

## 2. DEFINICIÓN DE ENDPOINTS

### 2.1. Autenticación: POST /api/auth/login

Se encarga de validar credenciales y devolver el JWT.

- **200 OK (Éxito)**

JSON

```
{
  "success": true,
  "data": {
    "token": "eyJhbGciOiJIUzI1NiIsIn...",
    "user": { "id": 1, "nombre": "Admin", "rol": "admin" }
  }
}
```

- **400 Bad Request (Validación fallida)** Faltan campos (ej. email o password vacíos). El mensaje indicará qué campo falta.
- **401 Unauthorized (Token expirado o credenciales inválidas)** {"success": false, "error": "Unauthorized", "message": "Correo o contraseña incorrectos."}
- **500 Internal Server Error** Error de conexión con la base de datos al buscar al usuario.

### 2.2. Catálogo: GET /api/productos

Devuelve la lista de productos disponibles. Soporta query params para búsquedas (?q=nombre).

- **200 OK (Éxito)** (Nota: Si no hay productos, devuelve un array vacío [], NO un error 404).

JSON

```
{
  "success": true,
  "data": [
    { "id": 1, "nombre": "Coca Cola 600ml", "precio": 18.50 }
  ]
}
```

- **400 Bad Request** Si se envían parámetros de búsqueda inválidos (ej. ?limit=-5).
- **500 Internal Server Error** Fallo interno en el servidor.

### 2.3. Comercios: GET /api/tiendas

Devuelve el listado de tiendas registradas en la plataforma.

- **200 OK (Éxito)**

JSON

```
{
  "success": true,
  "data": [
    { "id": 101, "nombre": "Abarrotes Doña Mary", "estado": "abierto" }
  ]
}
```

- **404 Not Found** Se usará exclusivamente si se busca una tienda por ID específico (GET /api/tiendas/:id) y esta no existe en la base de datos. {"success": false, "error": "Not Found", "message": "La tienda solicitada no existe."}

## ESTÁNDAR GLOBAL DE MANEJO DE ERRORES (SPA)

### 1. ARQUITECTURA DE ESTADOS EN FRONTEND

Toda petición de datos (Fetch/Axios) debe manejar un ciclo de vida basado en 4 estados. El estado estándar a utilizar en los componentes será: const [status, setStatus] = useState("idle")

- idle: Componente montado, ninguna petición en curso.

- loading: Petición enviada. Se muestra loader accesible (aria-busy="true" y texto visible).
- success: Datos recibidos (HTTP 200). Se renderiza la UI principal.
- error: Fallo detectado. Se muestra feedback y botón de reintento.

## 2. PROTOCOLO DE REACCIÓN ANTE ERRORES

El Frontend deberá implementar validaciones para detectar y reaccionar a los siguientes 4 escenarios críticos:

### A. Error de Red (Sin Internet)

- **Detección:** El navegador reporta desconexión (!navigator.onLine) o la promesa de fetch arroja un TypeError: Failed to fetch.
- **Reacción Frontend:**
  - Pausar peticiones.
  - Mostrar icono de "WiFi desconectado".
  - Mostrar alerta visual: "No hay conexión a internet. Verifica tu red."
  - Mostrar botón: **"Reintentar"**.

### B. Timeout (Tiempo de espera agotado)

- **Detección:** La petición al Backend supera los 10 segundos sin respuesta (se debe usar AbortController en el Frontend para forzar el corte).
- **Reacción Frontend:**
  - Ocultar el loader.
  - Mostrar alerta: "La petición está tardando demasiado. El servidor podría estar saturado."
  - Mostrar botón: **"Volver a intentar"**.

### C. Token Expirado (HTTP 401)

- **Detección:** El Backend responde con Status 401 Unauthorized en rutas protegidas.
- **Reacción Frontend (Flujo automático):**
  1. Limpiar el token inválido del localStorage o sessionStorage.
  2. Redirigir forzosamente al usuario a la pantalla de /login.
  3. Mostrar mensaje Toast (aria-live="assertive"): "Tu sesión ha expirado por seguridad. Vuelve a iniciar sesión."

**D. Error 500 Backend (Fallo del Servidor)**

- **Detección:** El Backend atrapa una excepción y responde con Status 500 Internal Server Error.
- **Reacción Frontend:**
  - Leer la propiedad message del JSON del Backend (ej. "Ocurrió un error inesperado en el servidor").
  - Mostrar ilustración o icono de "Mantenimiento/Error".
  - Mostrar botón: **"Recargar página"**.